

```
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
```

```
#define MAXPAROLA 30
#define MAXRIGA 80
```

```
int main(int argc, char *argv[])
{
    int freq[MAXPAROLA]; /* vettore di contatori
delle frequenze delle lunghezze delle parole */
    char riga[MAXRIGA];
    int i, inizio, lunghezza;
    FILE *f;
```

```
for(i=0; i<MAXPAROLA; i++)
    freq[i]=0;
```

```
if(argc != 2)
```

```
{
    fprintf(stderr, "ERRORE, serve un parametro con il nome del file\n");
    exit(1);
}
```

```
f = fopen(argv[1], "r");
if(f==NULL)
```

```
{
    fprintf(stderr, "ERRORE, impossibile aprire il file %s\n", argv[1]);
    exit(1);
}
```

```
while( fgets( riga, MAXRIGA, f ) != NULL )
```



Inter-Process Communication

Message Queues

Stefano Quer

Dipartimento di Automatica e Informatica

Politecnico di Torino

License Information

This work is licensed under the license



Attribution-NonCommercial-NoDerivatives 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to copy and distribute the material in any medium or format in unadapted form and for noncommercial purposes only.

① **BY:** Credit must be given to you, the creator.

② **NC:** Only noncommercial use of your work is permitted.

Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.

③ **ND:** No derivatives or adaptations of your work are permitted.

To view a copy of the license, visit:

<https://creativecommons.org/licenses/by-nc-nd/4.0/?ref=chooser-v1>

Introduction

- ❖ FIFOs (and pipes) are used to pass streams of anonymous bytes
 - Applications using FIFOs have to manage their own data chunking
 - They have to agree on data delimiters, such as end-of-field, end-of-record, etc.
- ❖ To pass structured data chunks it is necessary to use message queues
 - A **message queue** is a linked list of **messages**
 - Messages are stored within the kernel
 - The queue is identified through an identification **key**

Message !

Key !

Messages

What is a message?

- ❖ Message queues manipulate messages
 - Each message is composed must to be defined by the user as a C data structure including
 - The message type **mtype**
 - The data field **mtext** of maximum size N_BYTES

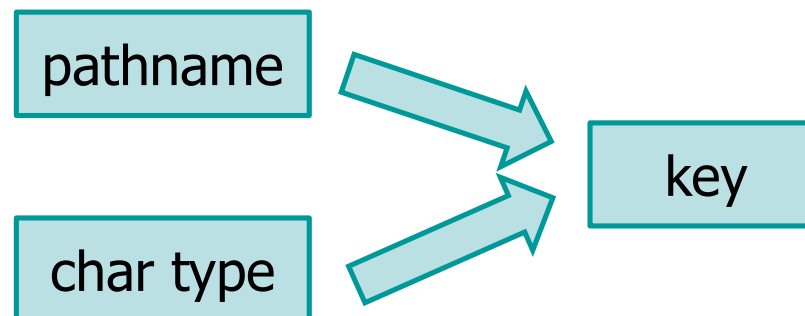
```
struct mesg_buffer {  
    long int mtype;  
    char mtext[N_BYTES];  
} message;
```

- Each new message is placed at the end of the queue

Keys

What is a key?

- ❖ Keys identify IPC structures
 - Have datatype **key_t**
 - Are defined as a long integer in header `sys/types.h`
- ❖ To generate an IPC key we can use function **ftok**
 - Function **ftok** converts a path and an integer value (in the range `[0,255]`) into a key
 - To share a key, the different processes (e.g., the clients and the server) must agree on the path and the value



Logic flow

❖ Logic flow to use a message queue

- Generate an IPC key we can use function **ftok**



- Create or open a message queue with **msgget**
- Send new messages with **msgsnd**
- Fetch messages from the queue with **msgrcv**
- Possibly, control the queue with **msgctl**

Function `ftok`

```
#include <sys/msg.h>
```

```
key_t ftok (const char *path, int id);
```

Return value
Message key, on success
The value -1, on error

❖ Parameters

- A standard (file) pathname (that **must** exist)
- A project ID (i.e., a char between 0 and 255)

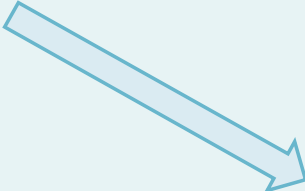
❖ The system call `ftok` generates a key from a pathname and a project ID

- Does not perform any sort of communication
- The key is used during the communication phase

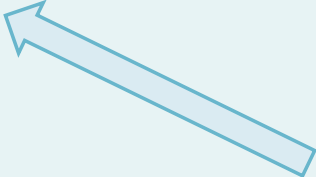
Operations

System Call	Meaning
<pre>int msgget (key_t key, int flag);</pre>	Creates or opens an existing message queue. The first parameter is a system key. The second one is a flag used to define the permission to a data structure associated to the message queue. The return value is the message queue identifier (msqid), on success, or the value -1, on error.
<pre>int msgctl (int msqid, int cmd, struct msqid_ds *buf);</pre>	Controls the queue, performs various operations on it. Parameter cmd (IPC_STAT, IPC_SET, IPC_RMID) specifies the command to be performed on the queue. The queue is specified by its identifier (msqid). The parameter msqid is the value returned by msgget.

Operations

System Call	Meaning
<pre>int msgsnd (int msqid, const void *ptr, size_t nbytes, int flag);</pre> 	Sends new messages. A new message is added to the end of a queue. The identifier msqid specifies the queue on which to send a message. The ptr argument points to the specific defined message data structure mymesg. nbytes specify the size of the data array in mymesg. The flag value is 0 or IPC_NOWAIT (to deal with error on full queues).

Operations

System Call	Meaning
<pre>ssize_t msgrcv (int msqid, void *ptr, size_t nbytes, long type, int flag);</pre> 	Fetches messages from a queue. Messages do not have to be fetched in a first-in, first-out order but can be fetched based on their type field. Parameters are the same than for msgsnd. Type argument specifies which message we want. The first message in the queue (type=0), the first message in the queue whose message type equals type is returned (type>0), etc.

Return value
The value 0, on success
The value -1, on error

Example

- ❖ Use a message queue to transfer a structured message

```
struct mesg_buffer {  
    long int mtype;  
    char mtext[N_BYTES];  
} message;
```

- ❖ From a sender (producer) to a receiver (consumer)



Example

The Writer

1 Reader + 1 Writer

(W) P_1

(R) P_2

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/msg.h>

#define L 512

struct msg_buffer {
    long msg_type;
    char msg_text[L];
} message;
```

Solution

The Writer

```
int main() {  
    key_t key;  
    int msgid;  
  
    key = ftok ("progfile", 65);  
    msgid = msgget (key, 0666 | IPC_CREAT);  
    message.mesg_type = 1;  
  
    printf ("Read data: ");  
    fgets (message.mesg_text, L, stdin);  
  
    msgsnd (msgid, &message, L*sizeof(char), 0);  
    printf ("Data send: %s\n", message.mesg_text);  
    return 0;  
}
```

Get key

(W) P₁

Get the id

Read message
from stdin

Send message

(R) P₂

Solution

The Reader

```
struct mesg_buffer {  
    long mesg_type;  
    char mesg_text[L];  
} message;
```

```
int main() {  
    key_t key;  
    int msgid;  
    key = ftok ("progfile", 65);  
    msgid = msgget (key, 0666 | IPC_CREAT);  
    msgrcv (msgid, &message, L*sizeof(char), 1, 0);  
    printf ("Data received: %s\n", message.mesg_text);  
    msgctl (msgid, IPC_RMID, NULL);  
    return 0;  
}
```

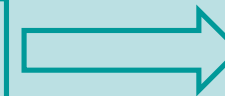
Get key

Get the id

Receive message

Remove the queue
from the system

(W) P₁



(R) P₂